

目录

Tricky	1
MillerRabin	1
行列式求值	2
分治 FFT	4
原根	5
质因数分解	7
Bsgs	7
拉格朗日插值	9
Lucas	9
快速沃尔什变换	11
子集卷积	12

Tricky

1. lcm 卷积

接下来考虑卷积怎么做。假设我们要求 $h(x) = f(x) \times g(x)$ ，卷积是 lcm 卷积；我们构造 $F(n) = \sum_{d|n} f(d)$ 。有结论：

$H(x) = F(x) \cdot G(x)$ 这里是点值对应相乘。然后我们根据莫比乌斯反演有 $f(n) = \sum_{d|n} \mu\left(\frac{n}{d}\right) F(d)$ 可以反演出 $h(x)$ 。单次卷积朴素调和级数是 $O(n \ln n)$ ，使用狄利克雷前缀和是 $O(n \ln \ln n)$ 。

gcd 卷积反过来

2. 0/1 分数规划

3. 整除分块

向上取整，从 r 推出 l ， $l = \left\lceil \frac{n}{\frac{n}{r}} \right\rceil$ 向下取整，从 l 推出 r ， $r = \left\lfloor \frac{n}{\frac{n}{l}} \right\rfloor$

4. dp 套 dp

5. dp 通过性质减少有效状态和转移

6. 树上背包， $sz_u + sz_v$ 优化转移

7. wqs 二分？

MillerRabin

```
const ll Prime[]={2,3,5,7,11,13,17,37};
const __int128 I=1;
ll Mul(ll x,ll y,ll p){return I*x*y%p;}
ll Pow(ll a,ll b,ll p){
```

```

ll r=1;
while(b){
    if(b&1){r=r*a%p;}
    a=a*a%p;b>>=1;
}
return r;
}
bool MillerRabin(ll n,ll a){
    ll d=n-1,r=0;
    while(!(d&1))d>>=1,r++;
    ll z=Pow(a,d,n);
    if(z==1)return true;
    for(int i=0;i<r;i++){
        if(z==n-1)return true;
        z=Mul(z,z,n);
    }
    return false;
}
bool isPrime(ll n){
    if(n<=1)return false;
    for(int i=0;i<8;i++){
        if(n==Prime[i])return true;
        if(n%Prime[i]==0)return false;
        if(!MillerRabin(n,Prime[i]))return false;
    }
    return true;
}

```

行列式求值

```

struct Matrix {
    static const int MAXN = 610;
    array<array<int, MAXN>, MAXN> data{};
    int n;
    array<int, MAXN>& operator[](int i) { return data[i]; }
};
inline int add(int x, int y) { int t = x + y; return t >= Mod ? t - Mod : t; }
inline int sub(int x, int y) { int t = x - y; return t < 0 ? t + Mod : t; }
inline int mul(int x, int y) { return int(1LL * x * y % Mod); }
int determinant(Matrix mat) {
    int det = 1;
    for(int i=0;i<=mat.n;i++){//未知元
        for(int j=i+1;j<=mat.n;j++){//方程
            while(mat[i][j]){
                int div=mat[j][i]/mat[i][i];
                for(int k=i;k<mat.n;k++)
                    mat[j][k]=sub(mat[j][k],mul(div,mat[i][k]));
                for(int k=i;k<mat.n;k++)
                    std::swap(mat[j][k],mat[i][k]);
                det=mul(det,Mod-1);
            }
        }
    }
}

```

```

        for(int k=i;k<=mat.n;k++)
            std::swap(mat[j][k],mat[i][k]);
        det=mul(det,Mod-1);
    }
}
for(int i=0;i<mat.n;i++)
    det=mul(det,mat[i][i]);

return det;
}

struct Matrixes{
    double Arr[550][550];int n,m;
    inline double* operator[](const int&idx){return Arr[idx];}
};
Matrixes A;
int n;
int main(){
    scanf("%d",&n);
    for(int i=1;i<=n;++i)
        for(int j=1;j<=n+1;j++)
            if(scanf("%lf",&A[i][j]) != 1) return 0;
    int now=1;
    for(int i=1;i<=n;i++){
        int maxi=now;
        for(int j=now+1;j<=n;j++)
            if(abs(A[j][i]) > abs(A[maxi][i]))
                maxi=j;
        if(abs(A[maxi][i]) < EPS) continue;
        for(int j=1;j<=n+1;j++)
            swap(A[now][j],A[maxi][j]);
        double pivot = A[now][i];
        for(int j=i; j<=n+1; j++)
            A[now][j] /= pivot;
        for(int j=1;j<=n;j++){
            if(j==now)continue;

            double tmp=A[j][i];

            for(int k=i;k<=n+1;k++)
                A[j][k]-=A[now][k]*tmp;
        }
        ++now;
    }
    int rank = now - 1;
    for(int i = rank + 1; i <= n; i++){
        if(fabs(A[i][n+1]) > EPS)
            return puts("-1"),0;
    }
    if(rank < n)
        return puts("0"),0;
    for(int i=1;i<=n;i++){
        printf("x%d=%.21f\n",i,A[i][n+1]/A[i][i]);
    }
}

```

```

    return 0;
}

```

分治 FFT

```

void InitNtt(const int&L){
    for(int i=1,t=1;i<L;i<=1,t++){
        Pw[t]=Pow(g,(Mod-1)/(i<<1));
        IPw[t]=Pow(Pw[t],Mod-2);
    }
}

void Ntt(vector<int>&Ar,int lim,int len,bool flag=false){
    vector<int>Rev(lim);
    for(int i=1;i<lim;i++){
        Rev[i]=(Rev[i>>1]>>1)|((i&1)<<(len-1));
    }
    for(int i=1;i<lim;i++){
        if(i<Rev[i])std::swap(Ar[i],Ar[Rev[i]]);
    }
    for(int i=1,t=1;i<lim;i<=1,t++){
        int T=!flag?Pw[t]:IPw[t];
        for(int j=0;j<lim;j+=(i<<1)){
            for(int k=0,w=1;k<i;k++,w=M(W,T)){
                int x=Ar[j+k],y=M(W,Ar[j+k+i]);
                Ar[j+k]=A(x,y);
                Ar[j+k+i]=A(x,N(y));
            }
        }
    }
    if(!flag)return ;
    int I=Pow(lim,Mod-2);
    for(int i=0;i<lim;i++)Ar[i]=M(Ar[i],I);
}

vector<int>F(200000),G(200000),AA(200000),BB(200000);
void Cdq(int L,int R,int len){
    if(L==R)return ;
    int MM=(L+R)>>1;
    Cdq(L,MM,len-1);
    int lim=(1<<len);
    std::copy(F.begin()+L,F.begin()+MM+1,AA.begin());
    std::fill(AA.begin()+MM-L+1,AA.begin()+lim,0);
    std::copy(G.begin(),G.begin()+R-L+1,BB.begin());
    Ntt(AA,lim,len);Ntt(BB,lim,len);
    for(int i=0;i<lim;i++)BB[i]=M(AA[i],BB[i]);
    Ntt(BB,lim,len,true);
    for(int i=MM+1;i<=R;i++)F[i]=A(F[i],BB[i-L]);
    Cdq(MM+1,R,len-1);
}

int main(){
    int n;
    InPut(n);
    for(int i=1;i<=n-1;i++)
        InPut(G[i]);
}

```

```

F[0]=1;
int len=0,lim=1;
while(lim<n-1)lim<=1,len++;
InitNtt(lim);
Cdq(0,lim-1,len);
for(int i=0;i<=n-1;i++)
    OutPut(F[i],' ');
return 0;
}

```

原根

```

int Phi[MaxN],Prime[MaxN],tot;
bool NotPrime[MaxN],Root[MaxN];
inline void Init(const int&limit){
    Phi[1]=1;
    for(int i=2;i<=limit;i++){
        if(!NotPrime[i]){
            Prime[++tot]=i;
            Phi[i]=i-1;
        }
        for(int j=1;j<=tot;j++){
            if(i*Prime[j]>limit)break;
            NotPrime[i*Prime[j]]=true;
            if(i%Prime[j]==0){
                Phi[i*Prime[j]]=Prime[j]*Phi[i];
                break;
            }
            Phi[i*Prime[j]]=Phi[i]*Phi[Prime[j]];
        }
    }
    Root[2]=Root[4]=true;
    for(int i=2;i<=tot;i++){
        for(int j=1;(1ll*j*Prime[i])<=limit;j*=Prime[i])Root[j*Prime[i]]=true;
        for(int j=2;(1ll*j*Prime[i])<=limit;j*=Prime[i])Root[j*Prime[i]]=true;
    }
}
inline int Gcd(int a,int b){return(b==0?a:Gcd(b,a%b));}
inline int GetPow(int x,int n,int Mod){
    int r=1;x=x%Mod;
    while(n){
        if(n&1)r=1ll*r*x%Mod;
        x=1ll*x*x%Mod;n>>=1;
    }
    return r;
}
int Fac[MaxN],cnt;
inline void GetFac(int x){
    for(int i=2;i*i<=x;i++){
        if(x%i==0){
            Fac[++cnt]=i;

```

```

        while(x%i==0){x/=i;}
    }
}
if(x>1)Fac[++cnt]=x;
}

inline bool Check(int x,int Mod){
    if(GetPow(x,Phi[Mod],Mod)!=1)return false;
    for(int i=1;i<=cnt;i++){
        if(GetPow(x,Phi[Mod]/Fac[i],Mod)==1)return false;
    }
    return true;
}

inline int MinRoot(int Mod){
    for(int i=1;i<Mod;i++){
        if(Check(i,Mod))return i;
    }
    return 0;
}

int Ans[MaxN],cans;
inline void GetRoot(int Mod,int x){
    int now=1;
    for(int i=1;i<=Phi[Mod];i++){
        now=(1ll*now*x)%Mod;
        if(Gcd(i,Phi[Mod])==1)Ans[++cans]=now;
    }
}

int main(){
    Init(1000000);
    int Task;scanf("%d",&Task);
    for(int Case=1;Case<=Task;Case++){
        int wtf,x;scanf("%d%d",&x,&wtf);
        if(Root[x]){
            cans=cnt=0;
            GetFac(Phi[x]);
            int mn=MinRoot(x);
            GetRoot(x,mn);
            sort(Ans+1,Ans+cans+1);
            printf("%d\n",cans);
            for(int i=1;i<=cans/wtf;i++){
                printf("%d ",Ans[i*wtf]);
            }
            puts("");
        }
        else{
            printf("%d\n",0);
            puts("");
        }
    }
    return 0;
}

```

质因数分解

```
inline long long PR(long long x){
    long long a,b,c,v,g;
    int i,j;
    while(true){
        a=b=rnd()%(x-1)+1;c=rnd()%(x-1)+1;
        v=1;
        i=0;j=1;
        while(++i){
            a=(One*a*a+c)%x;
            v=One*v*abs(a-b)%x;
            if(a==b||!v)break;
            if(!(i%63)||i==j){
                g=GetGcd(v,x);
                if(g>1)return g;
                if(i==j)b=a,j<=<1;
            }
        }
    }
}
long long maxi;
inline void Solve(long long x){
    if(x<=maxi||x<2)return ;
    if(IsPrime(x)){
        maxi=x;
        return ;
    }
    long long p=PR(x);
    while(x%p==0)x/=p;
    Solve(x),Solve(p);
}
long long n;
int main(){
    int Task;scanf("%d",&Task);
    while(Task--){
        scanf("%lld",&n);
        if(IsPrime(n))puts("Prime");
        else maxi=1,Solve(n),printf("%lld\n",maxi);
    }
    return 0;
}
```

Bsgs

```
#include<cstdio>
#include<cstring>
#include<algorithm>
#include<map>
#include<cmath>
```

```

using namespace std;

inline int Gcd(int a,int b){
    return b==0?a:Gcd(b,a%b);
}
inline void ExGcd(int a,int b,int &x,int &y){
    if(b==0){x=1,y=0;return ;}
    ExGcd(b,a%b,y,x);y-=(a/b)*x;
}
inline int GetInv(int a,int p){
    int x,y;ExGcd(a,p,x,y);
    return (x%p+p)%p;
}

map<int,int>H;
inline int Bsgs(int a,int b,int p){
    if(1%p==b%p)return 0;
    a%=p,b%=p;
    H.clear();
    int k=ceil(sqrt(1%p)),r=1;
    for(int t=0;t<k;t++){
        H[1ll*b*r%p]=t;
        r=1ll*r*a%p;
    }
    int ak=r;
    for(int t=1;t<=k;t++){
        auto q=H.find(r);
        if(q!=H.end())return 1ll*t*k-(q->second);
        r=1ll*r*ak%p;
    }
    return -1;
}
inline int ExBsgs(int a,int b,int p){
    if(1%p==b%p)return 0;
    a%=p;b%=p;
    int g=Gcd(a,p);
    if(g!=1){
        if(b%g!=0)return -1;
        int t=ExBsgs(a,1ll*(b/g)*GetInv(a/g,p/g)%(p/g),(p/g));
        return t== -1?-1:t+1;
    }
    return Bsgs(a,b,p);
}
int a,b,p;
int main(){
    while(~scanf("%d%d%d",&a,&p,&b)&&!(a==0&&b==0&&p==0)){
        int t=ExBsgs(a,b,p);
        if(t== -1)puts("No Solution");
        else printf("%d\n",t);
    }
    return 0;
}

```


拉格朗日插值

```
#include<stdio>
#include<string>
#include<algorithm>
const long long Mod=998244353;
inline long long Pow(long long base,long long index)
{
    long long power=1;base%=Mod;
    while(index!=0)
    {
        if(index&1)power=power*base%Mod;
        index>>=1;base=base*base%Mod;
    }
    return power;
}
long long Lagrange(long long *x,long long *y,long long n,long long k)
{
    long long ans=0;
    for(register int i=0;i<=n;i++)
    {
        long long s1=y[i],s2=1;
        for(register int j=0;j<=n;j++)if(i!=j)
        {
            s1=s1*(k-x[j]+Mod)%Mod;
            s2=s2*((x[i]-x[j]+Mod)%Mod)%Mod;
        }
        ans+=s1*Pow(s2,Mod-2)%Mod;
    }
    return (ans+Mod)%Mod;
}
long long n,k,x[21000],y[21000];
int main()
{
    scanf("%lld%lld",&n,&k);
    for(int i=0;i<n;i++)
        scanf("%lld%lld",&x[i],&y[i]);
    printf("%lld\n",Lagrange(x,y,n-1,k));
    return 0;
}
```

Lucas

```
inline void ExGcd(long long a,long long b,long long&x,long long&y){
    if(a==0)puts("Error");
    if(b==0){x=1,y=0;return ;}
    ExGcd(b,a%b,y,x);y-=(a/b)*x;
}
inline long long GetInv(long long a,long long p){
    long long x,y;
```

```

    ExGcd(a,p,x,y);
    return (x%p+p)%p;
}
inline long long GetPow(long long x,long long n,long long p){
    long long r=1;r%=p;
    while(n){
        if(n&1)r=r*x%p;
        x=x*x%p;n>>=1;
    }
    return r;
}
inline long long GetV(long long x,long long p){
    long long ans=0;
    while(x){ans+=(x/=p);}
    return ans;
}
long long Save[1100000];
inline long long GetR(long long x,long long p,long long mod){
    if(x==0)return 1;
    return GetR(x/p,p,mod)*GetPow(Save[mod-1],x/mod,mod)%mod*Save[x%mod]%mod;
}
inline long long CalcC(long long n,long long m,long long p,long long mod){
    long long c=GetV(n,p)-GetV(m,p)-GetV(n-m,p);
    long long ans=GetPow(p,c,mod);
    if(!ans)return 0;
    Save[0]=1;
    for(int i=1;i<mod;i++){
        if(i%p==0)Save[i]=Save[i-1];
        else Save[i]=Save[i-1]*i%mod;
    }
    ans=ans*GetR(n,p,mod)%mod;
    ans=ans*GetInv(GetR(m,p,mod),mod)%mod;
    ans=ans*GetInv(GetR(n-m,p,mod),mod)%mod;
    return ans;
}
long long res,tmod=1;
inline void Insert(long long now,long long mod){
    long long newmod=mod*tmod;
    res=(res*mod%newmod*GetInv(mod,tmod)%newmod+now*tmod%newmod*GetInv(tmod,mod)%newmod)%newmod;
    tmod=newmod;
}
long long n,m,p;
int main(){
    scanf("%lld%lld%lld",&n,&m,&p);
    for(long long i=2;i*i<=p;i++){
        if(p%i==0){
            long long mod=1;
            while(p%i==0){
                mod*=i;
                p/=i;
            }
            Insert(CalcC(n,m,i,mod),mod);
        }
    }
}

```

```

    }
    if(p>1)Insert(CalcC(n,m,p,p),p);
    printf("%lld\n",res);
    return 0;
}

```

快速沃尔什变换

```

inline void Copy(){
    for(int i=0;i<=limit-1;i++)TmpA[i]=Arr[i];
    for(int i=0;i<=limit-1;i++)TmpB[i]=Brr[i];
}
inline void FwtOr(int*Tmp,bool flag){
    for(int i=1;i<limit;i<<=1)
        for(int j=0;j<limit;j+=(i<<1))
            for(int k=0;k<i;k++){
                int x=Tmp[j+k],y=Tmp[j+k+i];
                Tmp[j+k]=x;
                Tmp[j+k+i]=flag?A(x,y):S(y,x);
            }
}
inline void FwtAnd(int*Tmp,bool flag){
    for(int i=1;i<limit;i<<=1)
        for(int j=0;j<limit;j+=(i<<1))
            for(int k=0;k<i;k++){
                int x=Tmp[j+k],y=Tmp[j+k+i];
                Tmp[j+k]=flag?A(x,y):S(x,y);
                Tmp[j+k+i]=y;
            }
}
inline void FwtXor(int*Tmp,bool flag){
    for(int i=1;i<limit;i<<=1){
        for(int j=0;j<limit;j+=(i<<1)){
            for(int k=0;k<i;k++){
                int x=Tmp[j+k],y=Tmp[j+k+i];
                Tmp[j+k]=M(flag?1:Inv2,A(x,y));
                Tmp[j+k+i]=M(flag?1:Inv2,S(x,y));
            }
        }
    }
}
int main(){
    scanf("%d",&n);
    limit=(1<<n);
    for(int i=0;i<=limit-1;i++)
        scanf("%d",&Arr[i]);
    for(int i=0;i<=limit-1;i++)
        scanf("%d",&Brr[i]);
    Copy();
    FwtOr(TmpA,1);FwtOr(TmpB,1);
}

```

```

for(int i=0;i<=limit-1;i++)TmA[i]=M(TmA[i],TmB[i]);
FwtOr(TmA,0);
for(int i=0;i<=limit-1;i++)printf("%d ",TmA[i]);
puts("");
Copy();
FwtAnd(TmA,1);FwtAnd(TmB,1);
for(int i=0;i<=limit-1;i++)TmA[i]=M(TmA[i],TmB[i]);
FwtAnd(TmA,0);
for(int i=0;i<=limit-1;i++)printf("%d ",TmA[i]);
puts("");
Copy();
FwtXor(TmA,1);FwtXor(TmB,1);
for(int i=0;i<=limit-1;i++)TmA[i]=M(TmA[i],TmB[i]);
FwtXor(TmA,0);
for(int i=0;i<=limit-1;i++)printf("%d ",TmA[i]);
puts("");
return 0;
}

```

子集卷积

```

inline void FMSub(int*Tmp,int limit,bool flag){
    for(int i=1;i<limit;i<=1)
        for(int j=0;j<limit;j++)
            if(j&i)TmA[j]=Add(TmA[j],flag?TmA[j-i]:Mod-TmA[j-i]);
}
int Arr[22][(1<<21)],Brr[22][(1<<21)],Ans[22][(1<<21)];
inline int PopCount(int x){return __builtin_popcount(x);}
int n,limit;
int main(){
    ReadInt(n);
    limit=1<<n;
    for(int i=0;i<=limit-1;i++)ReadInt(Arr[PopCount(i)][i]);
    for(int i=0;i<=limit-1;i++)ReadInt(Brr[PopCount(i)][i]);
    for(int i=0;i<=n;i++){
        FMSub(Arr[i],limit,1);
        FMSub(Brr[i],limit,1);
    }
    for(int i=0;i<=n;i++)
        for(int j=0;j<=i;j++)
            for(int k=0;k<=limit-1;k++)
                Ans[i][k]=Add(Ans[i][k],Mul(Arr[j][k],Brr[i-j][k]));
    for(int i=0;i<=n;i++){
        FMSub(Ans[i],limit,0);
    }
    for(int i=0;i<=limit-1;i++){
        printf("%d ",Ans[PopCount(i)][i]);
    }
    return 0;
}

```